Lab 4: VibeCoding - Chat History

Aryan Upadhyay | SRN: PES1UG25CS806 | Section C | Software Engineering (UE24CS341A)

Item	Detail
Assigned repo	https://github.com/SETAPESU26/59_rock_paper_scissor
AI tool	Claude (Claude Code extension in VS Code), model Claude Opus 5.5
Date	2 October 2026
Prompts used	2

This export records the AI-assisted session for Lab 4. The student's prompts are reproduced verbatim. The assistant's replies are recorded step by step: what it analysed, the code it wrote, and the checks it ran. The complete code diff is in the appendix.

Prompt 1 (student)

"complete all the labs and upload on my github"

Prompt 2 (student)

"give all the deliverables and uplaod them too"

Assistant - analysis

Cloned the repo and read README.md: one deliberate bug plus three features to add. Read main.py, game/game_engine.py and game/button.py. Recorded the original game before changing anything (videos/before_changes.mp4): the player throws ROCK against SCISSORS and the game says "You Lose! SCISSORS beats ROCK" and gives the point to the CPU.

Task 1 - Fix the inverted outcome bug

Cause: in determine_winner the rule table had every result reversed. Each winning pair for the player was mapped to "CPU" and each losing pair to "PLAYER":

```
rules = {
    ("ROCK", "SCISSORS"): "CPU",
    ("SCISSORS", "PAPER"): "CPU",
    ("PAPER", "ROCK"): "CPU",
    ("SCISSORS", "ROCK"): "PLAYER",
    ("PAPER", "SCISSORS"): "PLAYER",
    ("ROCK", "PAPER"): "PLAYER",
}
```

Fix: replaced the table with one map of what each move beats, so the winner comes from a single source of truth:

```
BEATS = {"ROCK": "SCISSORS", "SCISSORS": "PAPER", "PAPER": "ROCK"}

def determine_winner(self, player, cpu):
    if player == cpu:
        return "TIE"
    return "PLAYER" if BEATS[player] == cpu else "CPU"
```

Verification: all 9 combinations now give the standard result:

```
ROCK vs ROCK -> TIE           PAPER vs ROCK -> PLAYER        SCISSORS vs ROCK -> CPU
ROCK vs PAPER -> CPU          PAPER vs PAPER -> TIE          SCISSORS vs PAPER -> PLAYER
ROCK vs SCISSORS -> PLAYER    PAPER vs SCISSORS -> CPU       SCISSORS vs SCISSORS -> TIE
```

Task 2 - "First to X wins" match victory

- Added target_wins (default 3, shown in the HUD as "First to 3 wins").
- After each round the engine checks both scores; when one reaches the target, the round result stays on screen for 1.8 s and then the game enters a MATCH_OVER state.
- MATCH_OVER draws a dark overlay with a banner ("YOU WIN THE MATCH!" or "CPU WINS THE MATCH!"), the final score and "Press R to play again". Choice buttons are disabled until R is pressed; R calls reset_match().

Task 3 - Adaptive AI pattern tracking

- The engine keeps the player's last 6 throws in a deque.
- cpu_pick() gives every move a base weight of 1 and adds 1.5 x (how often the player threw X) to the move that beats X, then calls random.choices with those weights.
- The CPU decides before the current throw is added to the history, so it cannot "see" the player's move.
- When one move appears 3 or more times, a hint is shown: "CPU has noticed you favour ROCK...".

Verification (3000 simulated picks each):

```
CPU picks after 6x ROCK:   {'PAPER': 2499, 'ROCK': 263, 'SCISSORS': 238}   (PAPER ~83%)
                           note: "CPU has noticed you favour ROCK..."
CPU picks with no history: {'SCISSORS': 985, 'PAPER': 1021, 'ROCK': 994}   (uniform)
```

Task 4 - Procedural gesture icons and reveal animation

- New module game/icons.py draws the gestures with pygame primitives only: Rock (lumpy stone/fist with knuckle creases), Paper (ruled sheet with a folded corner) and Scissors (two blades crossing at a pivot, with red finger rings).
- Each round now has a COUNTDOWN state: the text cycles "ROCK... PAPER... SCISSORS..." (350 ms per beat) while both fists bob up and down in sync.
- At the end of the countdown both real gestures are revealed with a short scale-up "pop" and the result is scored.
- The choice buttons show a small icon next to the label. The window was enlarged to 640x400 to fit the arena.

Videos

Both videos are 10.5 s of real gameplay produced from the game's own render loop. A small recording script (headless pygame) sends real mouse-click and key events to GameEngine.handle_event on a fixed 30 fps clock and encodes each rendered frame to MP4. before_changes.mp4 shows wins being scored as losses (final score 1-4 after 4 winning throws). after_changes.mp4 shows the countdown, the icons, a correct win, the adaptive-AI hint after repeated ROCK throws, the match-over banner and the R restart.

Appendix - code diff (main.py, game/button.py, game/game_engine.py)

game/icons.py is a new file and is included in full in the repository.

```
diff --git a/game/button.py b/game/button.py
index 6da7b18..f954864 100644
--- a/game/button.py
+++ b/game/button.py
@@ -1,4 +1,5 @@
 import pygame
+from game.icons import draw_gesture


 class ChoiceButton:
@@ -12,18 +13,28 @@ class ChoiceButton:
     def contains(self, pos):
         return self.rect.collidepoint(pos)

-    def render(self, surface):
+    def render(self, surface, enabled=True):
         mouse_pos = pygame.mouse.get_pos()
-        bg_color = self.hover_color if self.rect.collidepoint(mouse_pos) else self.color
+        hovered = enabled and self.rect.collidepoint(mouse_pos)
+        bg_color = self.hover_color if hovered else self.color
+        if not enabled:
+            bg_color = tuple(c // 2 for c in self.color)

         pygame.draw.rect(surface, bg_color, self.rect, border_radius=8)
         pygame.draw.rect(surface, (230, 230, 235), self.rect, width=2, border_radius=8)

+        # Small gesture icon on the left, label on the right
+        icon_size = self.rect.height - 16
+        icon_center = (self.rect.left + 12 + icon_size // 2, self.rect.centery)
+        draw_gesture(surface, self.choice_name, icon_center, icon_size)
+
         text_surf = self.font.render(self.choice_name, True, (255, 255, 255))
+        text_left = self.rect.left + 20 + icon_size
+        text_area = self.rect.right - text_left
         surface.blit(
             text_surf,
             (
-                self.rect.centerx - text_surf.get_width() // 2,
+                text_left + (text_area - text_surf.get_width()) // 2,
                 self.rect.centery - text_surf.get_height() // 2,
             ),
         )
diff --git a/game/game_engine.py b/game/game_engine.py
index 2dcdc87..7b7437d 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -1,18 +1,33 @@
+import math
 import random
+from collections import Counter, deque
+
 import pygame
 from game.button import ChoiceButton
+from game.icons import draw_gesture
+
+# Which move beats which: key beats value
+BEATS = {"ROCK": "SCISSORS", "SCISSORS": "PAPER", "PAPER": "ROCK"}
+# The move that beats the key
+COUNTER = {loser: winner for winner, loser in BEATS.items()}
+
+STATE_IDLE = "IDLE"
+STATE_COUNTDOWN = "COUNTDOWN"
+STATE_RESULT = "RESULT"
+STATE_MATCH_OVER = "MATCH_OVER"


 class GameEngine:
-    def __init__(self, width, height):
+    def __init__(self, width, height, target_wins=3):
         self.width = width
         self.height = height

         self.choices = ["ROCK", "PAPER", "SCISSORS"]
-        btn_w, btn_h = 130, 50
+        btn_w, btn_h = 160, 50
         gap = 20
         total_w = 3 * btn_w + 2 * gap
         start_x = (width - total_w) // 2
-        btn_y = height - 85
+        btn_y = height - 70

         self.buttons = [
             ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50, 50), (200, 70,
    70)),
@@ -20,6 +35,28 @@ class GameEngine:
```

```
            ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (180,
    140, 30), (220, 180, 50)),
        ]

+       # Match settings
+       self.target_wins = target_wins
+
+       # Adaptive AI settings: look at the player's last N throws
+       self.history_size = 6
+       self.bias_strength = 1.5
+
        self.display_duration = 1800
        self.countdown_beat = 350
        self.countdown_words = ["ROCK...", "PAPER...", "SCISSORS..."]
        self.countdown_duration = self.countdown_beat * len(self.countdown_words)
        self.reveal_pop = 220
+
+       self.font_title = pygame.font.SysFont(None, 36)
+       self.font_hud = pygame.font.SysFont(None, 26)
+       self.font_arena = pygame.font.SysFont(None, 32)
+       self.font_small = pygame.font.SysFont(None, 22)
+       self.font_banner = pygame.font.SysFont(None, 54)
+
+       self.reset_match()
+
+   def reset_match(self):
        self.player_choice = None
        self.cpu_choice = None
        self.result_text = "Make your move!"
@@ -27,33 +64,45 @@ class GameEngine:

        self.player_score = 0
        self.cpu_score = 0
+       self.match_winner = None

-       self.round_resolved_time = 0
-       self.display_duration = 1800
-       self.showing_result = False
+       self.player_history = deque(maxlen=self.history_size)
+       self.ai_note = ""

-       self.font_title = pygame.font.SysFont(None, 36)
-       self.font_hud = pygame.font.SysFont(None, 26)
-       self.font_arena = pygame.font.SysFont(None, 32)
+       self.state = STATE_IDLE
+       self.state_started = pygame.time.get_ticks()

    def determine_winner(self, player, cpu):
        if player == cpu:
            return "TIE"
-
-       rules = {
-           ("ROCK", "SCISSORS"): "CPU",
-           ("SCISSORS", "PAPER"): "CPU",
-           ("PAPER", "ROCK"): "CPU",
-           ("SCISSORS", "ROCK"): "PLAYER",
-           ("PAPER", "SCISSORS"): "PLAYER",
-           ("ROCK", "PAPER"): "PLAYER",
-       }
-       return rules.get((player, cpu), "TIE")
+       return "PLAYER" if BEATS[player] == cpu else "CPU"
+
+   def cpu_pick(self):
+       """Pick a CPU move, biased toward countering the player's recent favourites."""
+       weights = {choice: 1.0 for choice in self.choices}
+       counts = Counter(self.player_history)
+       for move, count in counts.items():
+           weights[COUNTER[move]] += count * self.bias_strength
+
+       if counts:
+           favourite, fav_count = counts.most_common(1)[0]
+           if fav_count >= 3:
+               self.ai_note = f"CPU has noticed you favour {favourite}..."
+           else:
+               self.ai_note = ""
+
+       return random.choices(self.choices, weights=[weights[c] for c in self.choices])[0]

    def play_round(self, choice):
        self.player_choice = choice
-       self.cpu_choice = random.choice(self.choices)
+       # The CPU decides from history *before* this throw so it can't peek
+       self.cpu_choice = self.cpu_pick()
+       self.player_history.append(choice)
+
+       self.state = STATE_COUNTDOWN
+       self.state_started = pygame.time.get_ticks()

+   def resolve_round(self):
```

```
         outcome = self.determine_winner(self.player_choice, self.cpu_choice)
         if outcome == "PLAYER":
             self.player_score += 1
@@ -67,24 +116,95 @@ class GameEngine:
             self.result_text = f"It's a Draw! Both picked {self.player_choice}."
             self.result_color = (240, 210, 80)

-        self.showing_result = True
-        self.round_resolved_time = pygame.time.get_ticks()
+        if self.player_score >= self.target_wins:
+            self.match_winner = "PLAYER"
+        elif self.cpu_score >= self.target_wins:
+            self.match_winner = "CPU"
+
+        self.state = STATE_RESULT
+        self.state_started = pygame.time.get_ticks()

     def handle_event(self, event):
+        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
+            if self.state == STATE_MATCH_OVER:
+                self.reset_match()
+            return
+
         if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
+            if self.state not in (STATE_IDLE, STATE_RESULT):
+                return
             for btn in self.buttons:
                 if btn.contains(event.pos):
                     self.play_round(btn.choice_name)
                     break

     def update(self):
-        now = pygame.time.get_ticks()
-        if self.showing_result and (now - self.round_resolved_time >= self.display_duration):
+        elapsed = pygame.time.get_ticks() - self.state_started
+
+        if self.state == STATE_COUNTDOWN and elapsed >= self.countdown_duration:
+            self.resolve_round()
+        elif self.state == STATE_RESULT and self.match_winner and elapsed >= self.display_duration:
+            self.state = STATE_MATCH_OVER
+            self.state_started = pygame.time.get_ticks()
+        elif self.state == STATE_RESULT and elapsed >= self.display_duration:
             self.player_choice = None
             self.cpu_choice = None
             self.result_text = "Make your move!"
             self.result_color = (190, 195, 205)
-            self.showing_result = False
+            self.state = STATE_IDLE
+            self.state_started = pygame.time.get_ticks()
+
+    def render_arena(self, screen):
+        elapsed = pygame.time.get_ticks() - self.state_started
+        left_x, right_x, icon_y = self.width // 4, 3 * self.width // 4, 185
+        base_size = 100
+
+        you = self.font_hud.render("YOU", True, (100, 180, 255))
+        cpu = self.font_hud.render("CPU", True, (255, 120, 120))
+        screen.blit(you, (left_x - you.get_width() // 2, 100))
+        screen.blit(cpu, (right_x - cpu.get_width() // 2, 100))
+
+        vs = self.font_arena.render("VS", True, (120, 128, 145))
+        screen.blit(vs, (self.width // 2 - vs.get_width() // 2, icon_y - vs.get_height() // 2))
+
+        if self.state == STATE_COUNTDOWN:
+            # Both fists bob in sync once per beat before the reveal
+            beat_phase = (elapsed % self.countdown_beat) / self.countdown_beat
+            offset = int(-abs(math.sin(beat_phase * math.pi)) * 22)
+            draw_gesture(screen, "ROCK", (left_x, icon_y + offset), base_size)
+            draw_gesture(screen, "ROCK", (right_x, icon_y + offset), base_size)
+        elif self.player_choice and self.cpu_choice:
+            # Pop the revealed gestures in with a short scale-up
+            pop = min(1.0, elapsed / self.reveal_pop) if self.state == STATE_RESULT else 1.0
+            size = int(base_size * (0.6 + 0.4 * pop))
+            draw_gesture(screen, self.player_choice, (left_x, icon_y), size)
+            draw_gesture(screen, self.cpu_choice, (right_x, icon_y), size)
+        else:
+            for x in (left_x, right_x):
+                pygame.draw.circle(screen, (45, 52, 66), (x, icon_y), base_size // 2, width=3)
+                q = self.font_arena.render("?", True, (90, 98, 115))
+                screen.blit(q, (x - q.get_width() // 2, icon_y - q.get_height() // 2))
+
+    def render_match_over(self, screen):
+        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
+        overlay.fill((10, 12, 18, 215))
+        screen.blit(overlay, (0, 0))
+
+        won = self.match_winner == "PLAYER"
+        title = "YOU WIN THE MATCH!" if won else "CPU WINS THE MATCH!"
+        color = (80, 230, 120) if won else (240, 80, 80)
```

```
+
+        banner = pygame.Rect(40, self.height // 2 - 90, self.width - 80, 180)
+        pygame.draw.rect(screen, (30, 35, 46), banner, border_radius=14)
+        pygame.draw.rect(screen, color, banner, width=3, border_radius=14)
+
+        lines = [
+            (self.font_banner.render(title, True, color), banner.top + 28),
+            (self.font_arena.render(f"Final Score  {self.player_score} - {self.cpu_score}", True, (230,
    232, 238)), banner.top + 88),
+            (self.font_hud.render("Press R to play again", True, (190, 195, 205)), banner.top + 132),
+        ]
+        for surf, y in lines:
+            screen.blit(surf, (self.width // 2 - surf.get_width() // 2, y))

     def render(self, screen):
         screen.fill((24, 28, 36))
@@ -94,21 +214,29 @@ class GameEngine:

         p_surf = self.font_hud.render(f"Player Score: {self.player_score}", True, (100, 180, 255))
         c_surf = self.font_hud.render(f"CPU Score: {self.cpu_score}", True, (255, 120, 120))
+        t_surf = self.font_small.render(f"First to {self.target_wins} wins", True, (160, 166, 180))
         screen.blit(p_surf, (35, 52))
         screen.blit(c_surf, (self.width - c_surf.get_width() - 35, 52))
+        screen.blit(t_surf, (self.width // 2 - t_surf.get_width() // 2, 55))

         pygame.draw.line(screen, (45, 52, 66), (25, 82), (self.width - 25, 82), 2)

-        p_str = self.player_choice if self.player_choice else "--"
-        c_str = self.cpu_choice if self.cpu_choice else "--"
+        self.render_arena(screen)

-        arena_p = self.font_arena.render(f"Your Pick:  {p_str}", True, (225, 225, 230))
-        arena_c = self.font_arena.render(f"CPU Pick:  {c_str}", True, (225, 225, 230))
-        screen.blit(arena_p, (self.width // 2 - arena_p.get_width() // 2, 115))
-        screen.blit(arena_c, (self.width // 2 - arena_c.get_width() // 2, 155))
+        if self.state == STATE_COUNTDOWN:
+            beat = min(len(self.countdown_words) - 1, (pygame.time.get_ticks() - self.state_started) //
    self.countdown_beat)
+            res_surf = self.font_arena.render(self.countdown_words[beat], True, (230, 232, 238))
+        else:
+            res_surf = self.font_arena.render(self.result_text, True, self.result_color)
+        screen.blit(res_surf, (self.width // 2 - res_surf.get_width() // 2, 262))

-        res_surf = self.font_arena.render(self.result_text, True, self.result_color)
-        screen.blit(res_surf, (self.width // 2 - res_surf.get_width() // 2, 205))
+        if self.ai_note:
+            note = self.font_small.render(self.ai_note, True, (200, 150, 255))
+            screen.blit(note, (self.width // 2 - note.get_width() // 2, 296))

+        enabled = self.state in (STATE_IDLE, STATE_RESULT)
         for btn in self.buttons:
-            btn.render(screen)
+            btn.render(screen, enabled)
+
+        if self.state == STATE_MATCH_OVER:
+            self.render_match_over(screen)
diff --git a/main.py b/main.py
index c937edc..ffc6138 100644
--- a/main.py
+++ b/main.py
@@ -1,7 +1,7 @@
 import pygame
 from game.game_engine import GameEngine

-WIDTH, HEIGHT = 560, 360
+WIDTH, HEIGHT = 640, 400
 FPS = 60
```